

Maths For Games Report

up2193347

May 21, 2025

Contents

1	Artefact Overview	2
1.1	Renderer	2
1.2	Physics Engine	2
2	Maths Library	3
2.1	Vectors	3
2.2	Euler Angles & Quaternions	3
2.3	Matrices	6
2.4	Debugging	7
2.5	General Analysis	9
3	3D Renderer	9
3.1	Transformations & Projections	9
3.2	Meshes	9
3.2.1	Winding Order	9
3.3	Lighting	9
4	Physics Engine	12
4.1	Broad & Narrow Phase Collision	12
4.2	Sphere Overlap	12
4.3	Collision Response	13
4.4	Shooting Spheres	13
4.5	Reflection	13
4.6	Future Directions	17
5	References	18

1 Artefact Overview

1.1 Renderer

I created a basic OpenGL renderer capable of rendering 3D meshes, quads, and text. This required the implementation of projection and transformation matrices, the understanding of the representation of 3D meshes as vertices, normal vectors, and indices, and a basic lighting function.

1.2 Physics Engine

I created a custom physics engine based on *Game Physics In One Weekend* (Hodges, 2021a). It features sphere bodies that can collide realistically, and a codebase that can be extended to add further body types. The physics engine is integrated with the renderer to visualise the simulation, and provide an interface to add and remove objects from the simulation, adjust the timescale, and shoot spheres.

A 30 second video of the artefact running is viewable here:

<https://www.youtube.com/watch?v=HE2jYti2zx8>.

Figure 1

20 spheres created just after being created, beginning to fall to the ground.

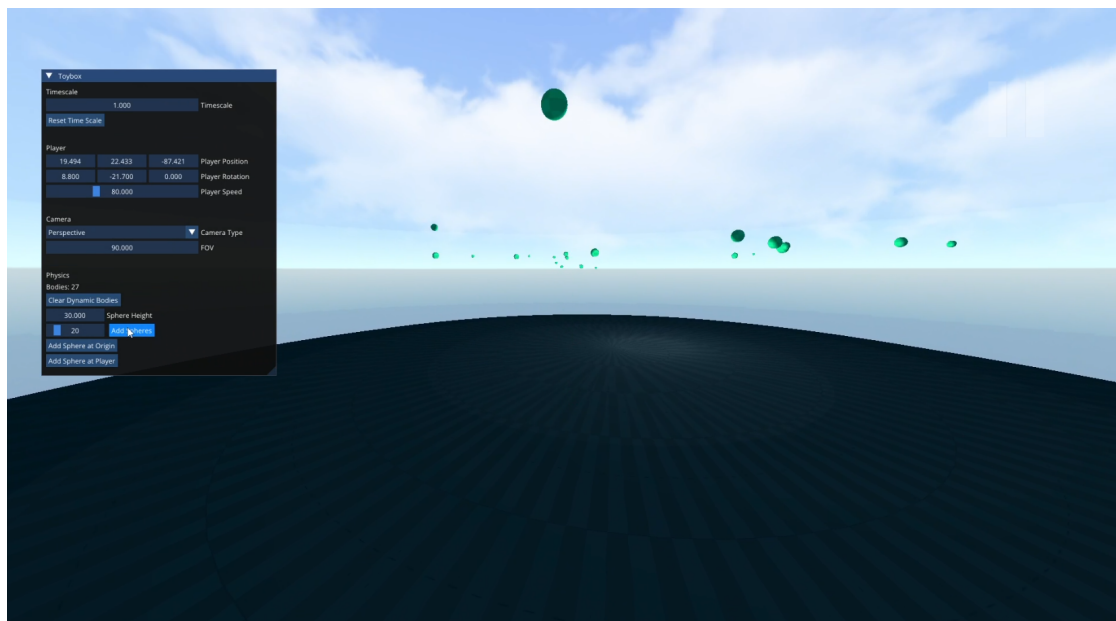
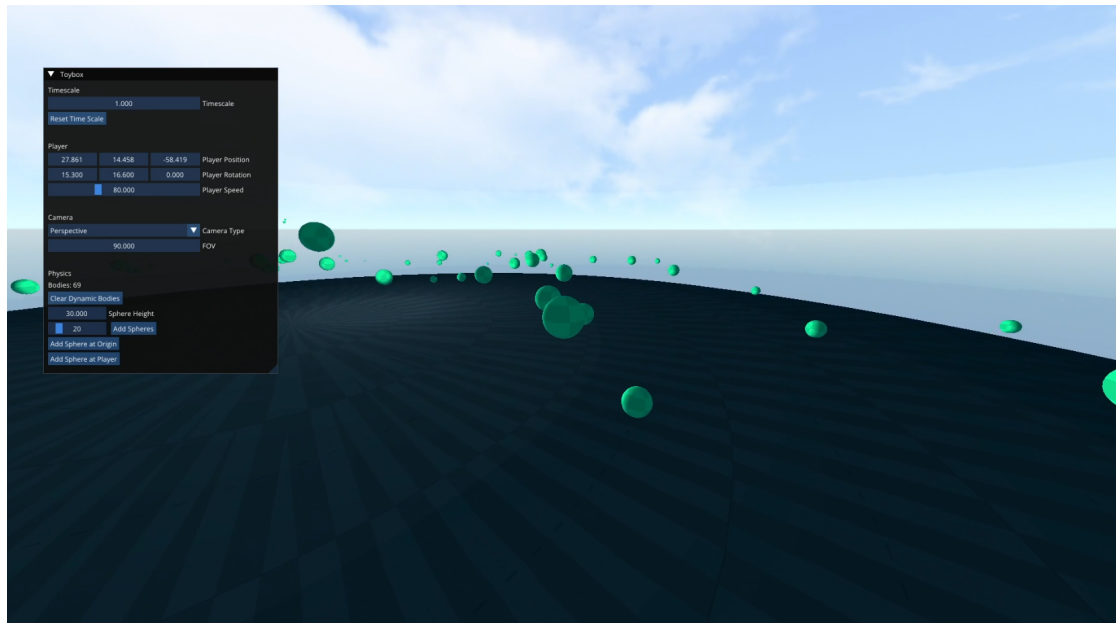


Figure 2

Spheres after falling and an initial bounce.



2 Maths Library

2.1 Vectors

The core of rendering and physics are **vectors**. When implementing vectors, I templated the backing type, such that a vector could be made of floats or doubles without having to copy all of the code. This could be useful if higher precision is needed, and allows for a compile-time toggle to increase physics accuracy at the cost of performance (as doubles take double the space in SIMD registers).

2.2 Euler Angles & Quaternions

I implemented both Euler angles and quaternions. Euler angles are much simpler to work with, as they represent rotations in a much more natural way, and can be converted to a rotation matrix in less operations. However, the problem of gimbal lock is much worse in the context of a physics engine, so quaternions are a must. I still use Euler angles in the renderer, so there are a lot of conversions between Euler and quaternions, and degrees and radians. An implementation that is made to only use quaternions may perform faster due to less conversions, and avoid some issues I had with missing a conversion.

Figure 3

Code to normalise a vector.

```
FORCEINLINE void Normalize()
{
    T length = Length();
    T invLength = 1.0f / length;
    if (!std::isinf(invLength)) // Check for division by zero (1 / 0 == inf). This makes normalisation safe for vectors with a length of 0.
    {
        X *= invLength;
        Y *= invLength;
        Z *= invLength;
    }
}

NODISCARD FORCEINLINE Vector3 Normalized() const
{
    Vector3 result = *this;
    result.Normalize();
    return result;
}
```

Figure 4

Code to get a vectors dot product.

```
// Calculate the dot product between this vector and another.
NODISCARD FORCEINLINE T Dot(const Vector3 &other, bool normalise = false) const
{
    if (normalise)
        return Normalized().Dot(other.Normalized(), normalise: false);

    return (X * other.X)
        + (Y * other.Y)
        + (Z * other.Z);
}
```

Figure 5

Code to convert quaternions to Euler angles.

```
// Based on glm
inline Rotator<T> EulerAngles() const
{
    Rotator<T> result;

    T const pitchY = static_cast<T>(2) * (Y * Z + W * X);
    T const pitchX = W * W - X * X - Y * Y + Z * Z;
    if (abs(pitchX) < 0.0001f && abs(pitchY) < 0.0001f) // avoid singularities
        result.Pitch = static_cast<T>(2) * atan2f(X, W);
    else
        result.Pitch = atan2f(pitchY, pitchX);

    result.Yaw = asin(MathUtil::Clamp(static_cast<T>(-2) * (X * Z - W * Y), static_cast<T>(-1), static_cast<T>(1)));

    T const rollY = static_cast<T>(2) * (X * Y + W * Z);
    T const rollX = W * W + X * X - Y * Y - Z * Z;
    if (abs(rollX) < 0.0001f && abs(rollY) < 0.0001f) // avoid singularities
        result.Roll = 0;
    else
        result.Roll = atan2f(rollY, rollX);

    result.Pitch = MathUtil::RadiansToDegrees(result.Pitch);
    result.Yaw = MathUtil::RadiansToDegrees(result.Yaw);
    result.Roll = MathUtil::RadiansToDegrees(result.Roll);

    return result;
}
```

Figure 6

Code to convert a Euler angle on an axis to a quaternion.

```
Quaternion(const Vector3<T> &axis, const float angleRadians)
{
    const float halfAngleRadians = angleRadians / 2;

    W = cosf(halfAngleRadians);

    const float halfAngleSine = sinf(halfAngleRadians);
    auto normalized = Vector3<T>() = axis.Normalized();
    X = normalized.X * halfAngleSine;
    Y = normalized.Y * halfAngleSine;
    Z = normalized.Z * halfAngleSine;
}
```

Figure 7

Code to convert between degrees and radians.

```
template <typename T>
    requires std::is_arithmetic_v<T>
NODISCARD FORCEINLINE static T DegreesToRadians(T degrees)
{
    return static_cast<T>(static_cast<double>(degrees) * 0.017453292519943295769236907684886);
}

template <typename T>
    requires std::is_arithmetic_v<T>
NODISCARD FORCEINLINE static T RadiansToDegrees(T radians)
{
    return static_cast<T>(static_cast<double>(radians) * 57.295779513082320876798154814105);
}
```

2.3 Matrices

Like vectors, my Matrix type is also templated, including the size of the Matrix (although it is limited to square matrices). I did this to reduce the amount of lines of code required to implement the types, however I believe the templating of the Size mainly increased complexity and hurt performance, as implementations could be less specialised and involved for loops. I avoided this slightly through the use of `if constexpr`, which are if statements that are evaluated at compile-time. If I were to template matrix sizes again, I would also template vector sizes so vectors could be used as the backing type, as the ability to perform cross products (for example) on columns would be useful.

My matrix type has functions for:

- Addition, subtraction, multiplication, and division of matrices
- Multiplication of vectors
- Transposing
- Inverting generally
- Finding the determinant
- Creating transformation and projection matrices

Figure 8

Code for multiplying 4x4 matrices.

```
else if constexpr (Size == 4)
{
    float a00 = n_Columns[0][0];
    float a01 = n_Columns[0][1];
    float a02 = n_Columns[0][2];
    float a03 = n_Columns[0][3];

    float a10 = n_Columns[1][0];
    float a11 = n_Columns[1][1];
    float a12 = n_Columns[1][2];
    float a13 = n_Columns[1][3];

    float a20 = n_Columns[2][0];
    float a21 = n_Columns[2][1];
    float a22 = n_Columns[2][2];
    float a23 = n_Columns[2][3];

    float a30 = n_Columns[3][0];
    float a31 = n_Columns[3][1];
    float a32 = n_Columns[3][2];
    float a33 = n_Columns[3][3];

    float b00 = other_n_Columns[0][0];
    float b01 = other_n_Columns[0][1];
    float b02 = other_n_Columns[0][2];
    float b03 = other_n_Columns[0][3];

    float b10 = other_n_Columns[1][0];
    float b11 = other_n_Columns[1][1];
    float b12 = other_n_Columns[1][2];
    float b13 = other_n_Columns[1][3];

    float b20 = other_n_Columns[2][0];
    float b21 = other_n_Columns[2][1];
    float b22 = other_n_Columns[2][2];
    float b23 = other_n_Columns[2][3];

    float b30 = other_n_Columns[3][0];
    float b31 = other_n_Columns[3][1];
    float b32 = other_n_Columns[3][2];
    float b33 = other_n_Columns[3][3];

    result[0][0] = (a00 * b00) + (a10 * b01) + (a20 * b02) + (a30 * b03);
    result[0][1] = (a01 * b00) + (a11 * b01) + (a21 * b02) + (a31 * b03);
    result[0][2] = (a02 * b00) + (a12 * b01) + (a22 * b02) + (a32 * b03);
    result[0][3] = (a03 * b00) + (a13 * b01) + (a23 * b02) + (a33 * b03);

    result[1][0] = (a00 * b10) + (a10 * b11) + (a20 * b12) + (a30 * b13);
    result[1][1] = (a01 * b10) + (a11 * b11) + (a21 * b12) + (a31 * b13);
    result[1][2] = (a02 * b10) + (a12 * b11) + (a22 * b12) + (a32 * b13);
    result[1][3] = (a03 * b10) + (a13 * b11) + (a23 * b12) + (a33 * b13);

    result[2][0] = (a00 * b20) + (a10 * b21) + (a20 * b22) + (a30 * b23);
    result[2][1] = (a01 * b20) + (a11 * b21) + (a21 * b22) + (a31 * b23);
    result[2][2] = (a02 * b20) + (a12 * b21) + (a22 * b22) + (a32 * b23);
    result[2][3] = (a03 * b20) + (a13 * b21) + (a23 * b22) + (a33 * b23);

    result[3][0] = (a00 * b30) + (a10 * b31) + (a20 * b32) + (a30 * b33);
    result[3][1] = (a01 * b30) + (a11 * b31) + (a21 * b32) + (a31 * b33);
    result[3][2] = (a02 * b30) + (a12 * b31) + (a22 * b32) + (a32 * b33);
    result[3][3] = (a03 * b30) + (a13 * b31) + (a23 * b32) + (a33 * b33);
}
```

A consistent issue I encountered with my Matrix class was mixing up rows and columns. My initial implementation of matrix multiplication had this issue, which I debugged mainly through trial and error. To avoid this in the future, I renamed my data variable from `Data` to `Columns`, which made it more explicit in the code that the Matrix data was column-major.

2.4 Debugging

When debugging, I used an online matrix calculator (Reshish, n.d.) to validate my results. As seen in Figure 10, I was able to validate the correctness of my 3x3 inversion function using a calculator, which was useful to find in which calculations errors were occurring.

In future, it would perhaps be easier to include an established maths library, such as GLM (G-Truc Creation, 2024), in the codebase such that it could be used as a reference for correct outputs; keeping validation within the code would allow the easy use of asserts and the creation of tests.

Figure 9

Code for finding the transpose of a matrix.

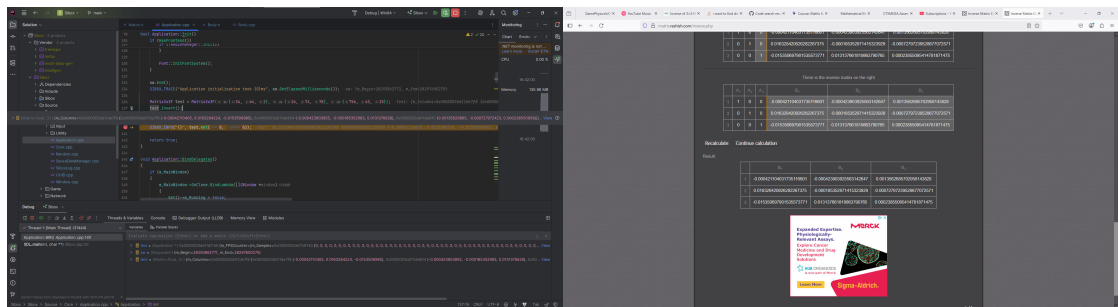
```
Matrix<T, Size> Transpose() const
{
    Matrix<T, Size> result;

    for (s32 i = 0; i < Size; i++)
        for (s32 j = 0; j < Size; j++)
            result.m_Columns[i][j] = m_Columns[j][i];

    return result;
}
```

Figure 10

Validating my Matrix 3x3 invert function using an online calculator.



2.5 General Analysis

The performance of the library could be improved heavily through the use of SIMD intrinsics, which allow you to operate on multiple pieces of data with one CPU instruction. This is mainly for vectors, as matrix multiplication benefits less from SIMD (Cui et al., 2019). However, this complicates the code significantly, so I decided it was out of scope for this project.

3 3D Renderer

3.1 Transformations & Projections

Vertex positions must be converted from model space to world space, and then to screen space. Graphics APIs require vertices to be in the -1 to 1 space, called normalised device coordinates.

Converting from model space to world space can be done through a transformation matrix. Implementing these were simple, however I took inspiration from GLM (G-Truc Creation, 2024) to optimise the creation of rotation matrices by merging the individual yaw, pitch, and roll matrices into one matrix creation, as seen in Figure 11.

To view these world space vertices through a camera and convert to NDC, we need a view-projection matrix. View matrices are covered by the previous translation matrices. Projections include perspective and orthographic. For a general 3D game, we need a perspective matrix, which is surprisingly simple to create, as seen in Figure 12.

3.2 Meshes

Meshes are represented as a list of vertices, all of which have a 3D vector representing their position, a 3D vector for their normal, and a 2D vector for their texture coordinate. An important point here was ensuring the in-memory layout of my vector classes are correct such that we can copy the vertex data directly into the arrays, to speed up loading.

3.2.1 Winding Order

To determine if a triangle is facing the camera or not, GPUs take the cross product of a triangle's first and second edge (Michel, 2016). If that cross product points towards the camera, it's a front face and will be rendered, but if it doesn't, it is discarded. Therefore, the indices of the mesh determine which way the faces face, as they change which edges are the first and second edges. I had issues with meshes being inverted, but this was fixed by flipping the indices.

3.3 Lighting

I implemented basic lighting based on the work of Phong (1975). Using the dot product of the light direction and world-space normal of a pixel, we can calculate the intensity of some lighting, and mix the light colour into the texture colour depending on that result.

Figure 11

Code for constructing a rotation matrix.

```
static Matrix<T, 4> MakeRotationMatrixSlow(T yawRadians, T pitchRadians, T rollRadians)
{
    return MakeYawRotationMatrix(yawRadians) * MakePitchRotationMatrix(pitchRadians) *
           MakeRollRotationMatrix(rollRadians);
}

// This is a faster version of the MakeRotationMatrix function, as it "unrolls" the creation and multiplication of
// the individual rotation matrices (yaw, pitch, roll) into a single matrix.
NODISCARD static Matrix<T, 4> MakeRotationMatrix(T yawRadians, T pitchRadians, T rollRadians)
{
    Matrix<T, 4> rotationMatrix;

    T yawCos = cos(yawRadians);
    T yawSin = sin(yawRadians);
    T pitchCos = cos(pitchRadians);
    T pitchSin = sin(pitchRadians);
    T rollCos = cos(rollRadians);
    T rollSin = sin(rollRadians);

    rotationMatrix[0][0] = yawCos * rollCos + yawSin * pitchSin * rollSin;
    rotationMatrix[0][1] = rollSin * pitchCos;
    rotationMatrix[0][2] = -yawSin * rollCos + yawCos * pitchSin * rollSin;
    rotationMatrix[0][3] = static_cast<T>(0);
    rotationMatrix[1][0] = -yawCos * rollSin + yawSin * pitchSin * rollCos;
    rotationMatrix[1][1] = rollCos * pitchCos;
    rotationMatrix[1][2] = rollSin * yawSin + yawCos * pitchSin * rollCos;
    rotationMatrix[1][3] = static_cast<T>(0);
    rotationMatrix[2][0] = yawSin * pitchCos;
    rotationMatrix[2][1] = -pitchSin;
    rotationMatrix[2][2] = yawCos * pitchCos;
    rotationMatrix[2][3] = static_cast<T>(0);
    rotationMatrix[3][0] = static_cast<T>(0);
    rotationMatrix[3][1] = static_cast<T>(0);
    rotationMatrix[3][2] = static_cast<T>(0);
    rotationMatrix[3][3] = static_cast<T>(1);

    return rotationMatrix;
}
```

Figure 12

Code for constructing a perspective projection matrix.

```
static Matrix<T, 4> MakePerspective(T verticalFov, T aspect, T near, T far)
{
    Matrix<T, 4> result(0);
    T const tanOfHalfVerticalFov = tan(verticalFov / static_cast<T>(2));
    result[0][0] = static_cast<T>(1) / (aspect * tanOfHalfVerticalFov);
    result[1][1] = static_cast<T>(1) / (tanOfHalfVerticalFov);
    result[2][2] = far / (far - near);
    result[2][3] = static_cast<T>(1);
    result[3][2] = -(far * near) / (far - near);
    return result;
}
```

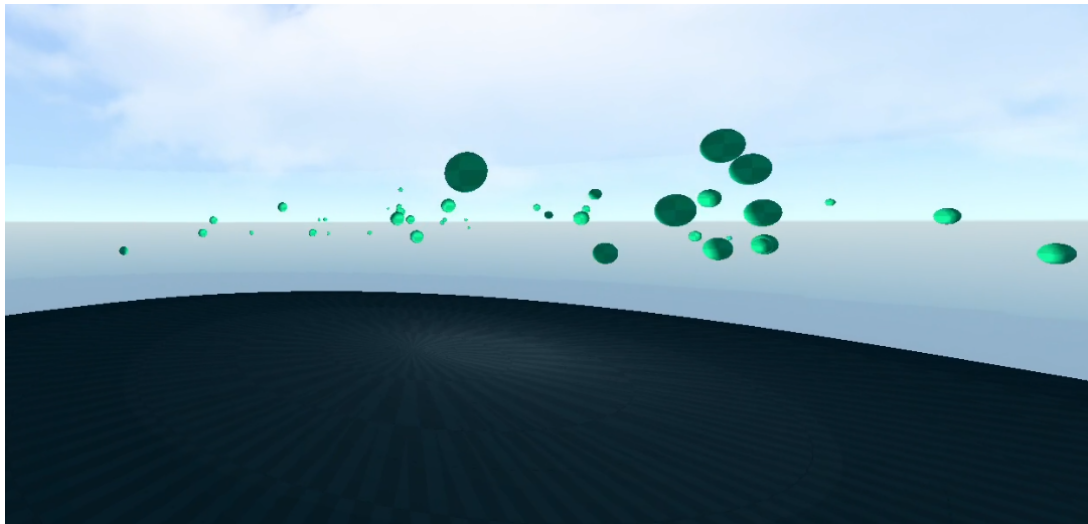
Figure 13

Shader code for basic "Phong" illumination.

```
vec3 norm = normalize(Input.Normal);  
vec3 lightDir = normalize(vec3(10, 10, 10) - Input.WorldPos);  
float diff = clamp(dot(norm, lightDir), 0.0, 0.7);  
vec3 diffuse = diff * vec3(1, 1, 1);  
  
float ambientStrength = 0.75;  
vec3 ambient = ambientStrength * vec3(0.02, 0.6, 0.851);  
  
o_Color = vec4((ambient + diffuse), 1) * texColor;
```

Figure 14

Objects are lit by a light positioned near the center of the world.



4 Physics Engine

4.1 Broad & Narrow Phase Collision

Advanced sphere overlaps are expensive to compute, so we use a "Sweep and Prune" algorithm to avoid full overlap checks for each pair. As seen in Figure 15, for each body, we find it's bounds, and expand them by the velocity of the body so it represents all space the object occupies over the physics step, including a small margin of error to account for floating point precision. We then use the dot product of a normalized $\begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$ vector and the bounds extreme vectors to create scalars that tell us if bounds intersect when compared (Department of Mathematics, Oregon State University, 1996). If a bounds minimum is above another bounds minimum but below its maximum, then they intersect. As seen in Figure 16, we can use this to only do expensive tests on pairs of bodies we know are likely to collide.

Figure 15

Code for creating and sorting the bounds of physics bodies.

```
void PhysicsScene::SortBodiesBounds(const Body **bodies, const s32 bodyCount, PseudoBody *sortedArray, const f32 delta)
{
    Vector3F axis = { scalar 1.0f };
    axis.Normalize();

    for (s32 i = 0; i < bodyCount; i++)
    {
        // Get the body and it's initial bounds.
        const Body *body = bodies[i];
        BoundsF bounds = body->GetShape().GetBounds(body->Position, body->Rotation);

        // Add the velocity to the bounds to account for the movement of the body.
        bounds.Expand(point bounds.Min + body->LinearVelocity * delta);
        bounds.Expand(point bounds.Max + body->LinearVelocity * delta);

        // Add a small epsilon to the bounds to make sure we catch possible collisions.
        constexpr float epsilon = 0.01f;
        bounds.Expand(point bounds.Min + Vector3F( scalar -1.0f ) * epsilon);
        bounds.Expand(point bounds.Min + Vector3F( scalar 1.0f ) * epsilon);

        // Fill in our pseudo body array.
        sortedArray[i * 2 + 0].ID = i;
        sortedArray[i * 2 + 0].Value = axis.Dot(bounds.Min);
        sortedArray[i * 2 + 0].IsMin = true;

        sortedArray[i * 2 + 1].ID = i;
        sortedArray[i * 2 + 1].Value = axis.Dot(bounds.Max);
        sortedArray[i * 2 + 1].IsMin = false;
    }

    // Sort the bodies.
    qsort(sortedArray, bodyCount * 2, sizeof(PseudoBody), CompareSAP);
}
```

4.2 Sphere Overlap

As seen in Figure 17, checking if two sphere's intersect is simple; we can simply check if the sum of their radii is less than the distance between the two spheres.

A more advanced intersection function will also tell us at what point in the physics step the objects collide. For this, we can simplify our check by taking the relative velocities of the spheres, which effectively makes one body static. Then, we can check for a collision between the static sphere and a capsule representing the space covered by the other sphere in the step. This can be simplified to a line trace and sphere test, with the new sphere having the combined radius of both spheres. The ray-sphere test can be implemented through a quadratic equation; if there is no solutions, there is no intersection, however if there are two, those two

solutions will also give us the points along the line that the collisions occurred.

4.3 Collision Response

With collisions detected, next they must be responded to. We can use the normal of the collision to send the bodies in the correct directions. We can define an "elasticity" for bodies to represent how much momentum is lost when a collision occurs. The formula for an elastic collision impulse is as follows:

$$J = \frac{-(1 + e)(\vec{v}_{rel} \cdot \hat{n})}{\frac{1}{m_1} + \frac{1}{m_2}}$$

As seen in Figure 18, this is easily translated to code.

4.4 Shooting Spheres

With all these systems in place, shooting spheres is easy. We simply create a sphere and give it an offset and velocity based on the camera's forward vector, calculated by rotating the global forward vector $\begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$ by the camera's rotation.

4.5 Reflection

I heavily used the debugger to understand how errors were being created in the physics simulation. I stepped through the frame and found which maths functions returned incorrect values. Another issue I had was with collision responses; the object would be moved outside of the collider, but floating point inaccuracies meant it was still very slightly within it, so when the pair was checked again, the collision response was undone. I fixed this by adding a tiny offset to the response. This was fixed separately when collision pairs were introduced.

Also, I have learned to not give functions "secret" operations; a bug was caused by my Cross product function normalising the vectors before calculating the cross product. This was configurable with a boolean parameter, but it has a default value of true, so I did not realise for a long time. I solved this by changing the default value to false.

Figure 16

Code for building likely collision pairs from the sorted "pseudo bodies".

```
void PhysicsScene::BuildPairs(std::vector<CollisionPair> &pairs, const PseudoBody *sortedBodies, const s32 bodyCount)
{
    pairs.clear();

    for (s32 i = 0; i < bodyCount * 2; i++)
    {
        const PseudoBody& pseudoBodyA = sortedBodies[i];
        if (!pseudoBodyA.IsValid())
            continue;

        CollisionPair pair;
        pair.BodyAID = pseudoBodyA.ID;

        for (s32 j = i + 1; j < bodyCount * 2; j++)
        {
            const PseudoBody& pseudoBodyB = sortedBodies[j];
            if (pseudoBodyB.ID == pseudoBodyA.ID)
            {
                // We've found the other side of the bounds of body A, so we're done looking for pairs for body A.
                break;
            }

            if (!pseudoBodyB.IsValid())
                continue;

            pair.BodyBID = pseudoBodyB.ID;
            pairs.push_back(pair);
        }
    }
}
```

Figure 17

Code for testing for an intersection of two spheres.

```
bool SphereIntersects(const Body &bodyA, const Body &bodyB)
{
    f32 radiusSum = bodyA.GetShape().GetRadius() + bodyB.GetShape().GetRadius();
    f32 distSquared = (bodyB.Position - bodyA.Position).LengthSquared();

    if (distSquared <= radiusSum * radiusSum)
        return true;

    return false;
}
```

Figure 18

Code for calculating and applying a collision impulse.

```
// Calculate the collision impulse.
const Vector3F velocityDiff = velocityA - velocityB;
const float impulse = (1.0f + elasticity) * velocityDiff.Dot(contact.NormalWorldSpace, normalise: false) / (bodyAInverseMass +
    bodyBInverseMass);
const Vector3F impulseVector = contact.NormalWorldSpace * impulse;

bodyA->ApplyImpulse(worldSpacePointOnA, impulse: -impulseVector);
bodyB->ApplyImpulse(worldSpacePointOnB, impulse: impulseVector);
```

Figure 19

Code for shooting a sphere.

```
if (Input::IsMouseButtonDownThisFrame(SIBOX_MOUSE_BUTTON_RIGHT))
{
    auto sphere :Ref<SphereEntity> = m_World->AddEntity<SphereEntity>("Sphere");
    sphere->Setup( radius: Random::Float( min: 0.5f, max: 2.f));
    Vector3F playerLookDirection = m_Player->GetCamera()->Transformation.GetForwardVector();
    Vector3F playerPosition      = m_Player->EntityTransform.Position;
    Vector3F spawnPosition      = playerPosition + playerLookDirection * 5.0f;
    sphere->SetBodyPosition(spawnPosition);
    sphere->GetBody()->LinearVelocity = playerLookDirection * 100.0f;
    m_DynamicBodies.push_back( Val: sphere.get());
}
```

Figure 20

Code to fix floating point imprecision issues with collision responses.

```
if (contact.TimeOfImpact == 0.0f)
{
    // We move the bodies apart, and the amount each body moves is based on their mass relative to the other body.
    const float aWeight = bodyA->InverseMass / (bodyA->InverseMass + bodyB->InverseMass);
    const float bWeight = bodyB->InverseMass / (bodyA->InverseMass + bodyB->InverseMass);

    // Penetration is a vector representing how far the two bodies are penetrating into each other.
    Vector3F penetration = contact.WorldSpacePointOnB - contact.WorldSpacePointOnA;

    // MW @todo @hack: add a small amount more to the weight such that we definitely avoid the intersection in the next check.
    // When we spawn a sphere at the origin and let it fall down the base sphere, the sphere will not bounce back up.
    // Stepping through the code, the first collision check works fine - the sphere gets it's linear velocity set to be
    // the reverse of it's current velocity (from gravity), meaning it should bounce up. However, when we check the pair
    // again (as we check each pair twice at the moment), they still technically collide due to floating point precision,
    // and the collision impulse is applied again in the opposite direction, meaning the sphere loses its velocity.
    penetration *= 1.005f;

    bodyA->Position += penetration * aWeight;
    bodyB->Position -= penetration * bWeight;
}
```

Figure 21

Code for testing for an intersection between a ray and a sphere, utilising a quadratic equation. It was the source of broken collisions, due to the hidden feature of cross product function normalising the vectors.

```
bool RayIntersectsSphere(const Vector3F &rayStart,
                        const Vector3F &rayDirection,
                        const Vector3F &sphereCenter,
                        const f32 sphereRadius,
                        float & t1,
                        float & t2)
{
    const Vector3F m = sphereCenter - rayStart;
    const f32 a = rayDirection.Dot(rayDirection);
    const f32 b = m.Dot(rayDirection);
    const f32 c = m.Dot(m) - sphereRadius * sphereRadius;

    const f32 delta = b * b - a * c;
    const f32 invA = 1.0f / a;

    if (delta < 0.0f)
    {
        // This is a quadratic equation - less than 0 means no real solutions.
        return false;
    }

    const f32 deltaRoot = sqrtf(delta);
    t1 = invA * (b - deltaRoot);
    t2 = invA * (b + deltaRoot);

    return true;
}
```

As seen in Figure 22, unlike Hodges (2021a), I used switch statements in place of virtual functions to improve performance. Lessening indirection is key to improving the performance of hot paths like Shape functions.

Figure 22

Switch statements used for shape getters, instead of the overridden virtual functions like Hodges (2021a).

```
BoundsF Shape::GetBounds(const Vector3F &position, const QuaternionF &orientation) const
{
    switch (m_Type)
    {
        case ShapeType::Sphere:
        {
            BoundsF output;
            output.Min = Vector3F(-GetRadius()) * position;
            output.Max = Vector3F(GetRadius()) * position;
            return output;
        }
        case ShapeType::AABB:
        case ShapeType::Invalid:
        default:
            STD_ERROR("Invalid shape type for inertia tensor calculation.");
            return {};
    }
}
```

I also found from reading the physics book that I should learn more about traditional math notation before delving further into this content, as it would make understanding the equations significantly easier.

4.6 Future Directions

In future, I'd be interested in adding more collision shapes, especially general convex shapes through the use of the GJK algorithm (Hodges, 2021b). Furthermore, I would like to implement some of the optimisations I have listed here, as performance is fairly poor for only spheres.

5 References

- Cui, C., Zhang, X., & Jin, Z. (2019). Performance Analysis of Existing SIMD Architectures. In W. Xu, L. Xiao, J. Li, & Z. Zhu (Eds.), *Computer Engineering and Technology* (pp. 34–47). Springer. https://doi.org/10.1007/978-981-15-1850-8_4
- Department of Mathematics, Oregon State University. (1996). *Dot Products and Projections*. Retrieved May 20, 2025, from <https://sites.science.oregonstate.edu/math/home/programs/undergrad/CalculusQuestStudyGuides/vcalc/dotprod/dotprod.html>
- G-Truc Creation. (2024, February 26). *GLM* (Version 1.0.1). Retrieved May 19, 2025, from <https://github.com/g-truc/glm>
- Hodges, G. (2021a, January 3). *Game Physics In One Weekend*.
- Hodges, G. (2021b, January 3). *Game Physics The Next Week*.
- Michel, C. (2016, February 13). *Understanding front faces - winding order and normals*. CMICHEL. Retrieved May 19, 2025, from <https://cmichel.io/understanding-front-faces-winding-order-and-normals>
- Phong, B. T. (1975). Illumination for computer generated pictures. *Commun. ACM*, 18(6), 311–317. <https://doi.org/10.1145/360825.360839>
- Reshish. (n.d.). *Inverse Matrix Calculator*. Retrieved May 19, 2025, from <https://matrix.reshish.com/inverse.php>